

DMS-技术复现报告

莫勋悦

2026 年 8 月 24 日

摘要

GUI Agent 在执行长期、多步骤任务时，难以有效利用历史交互经验，简单累积历史轨迹可能造成上下文膨胀和低质量经验干扰。Darwinian Memory System (DMS) 通过分层记忆、效用驱动选择和动态淘汰，使 Agent 能够保留高价值经验并逐步淘汰低价值经验，实现记忆的自我演化。本报告基于 AndroidWorld 和 Qwen3-VL-8B-Instruct 对 DMS 进行复现，实现其核心记忆机制，并通过多轮任务实验验证其有效性。

一、项目概述与复现目标

1. 问题背景

随着 LVLM 的发展，GUI Agent 已能通过点击、输入等操作完成多步骤任务。然而，在长期交互中，仅依赖当前上下文决策的 Agent 难以有效利用过往经验。

保存历史轨迹是一种直接方案，但存在明显问题：一是历史信息持续占用上下文，导致 Token 消耗增加；二是不同经验质量参差不齐，过时经验会干扰模型决策；三是 GUI 任务中的高层意图与低层操作具有不同粒度，完整轨迹难以灵活复用。

因此，一个有效的记忆系统不仅需要保存历史经验，还应能评估经验价值、选择性复用，并及时淘汰低价值信息。

2. Darwinian Memory System

Darwinian Memory System (DMS) [1] 针对上述问题提出了一种自调节记忆机制。其核心思想借鉴生物进化中的“适者生存”机制，将 Agent 在任务执行过程中产生的经验视为具有不同价值的记忆单元，并通过筛选和更新实现记忆演化。其包含三个核心机制：

- (1) **分层记忆 (Hierarchical Memory)**：将历史经验解耦成任务意图 (plan) 和具体动作 (traj) 进行组织，使不同粒度的经验能够分别存储、检索和组合。

- (2) **效用驱动的选择 (Utility-driven Selection)**: 根据任务执行过程中获得的反馈评估不同记忆单元的价值, 给更高价值的经验更高的保留优先级。
- (3) **动态淘汰 (Dynamic Pruning)**: 随着记忆库不断增长, 系统利用 Elbow 方法移除低价值记忆, 使记忆规模保持在可控范围, 同时减少无效历史信息对模型决策的干扰。

通过上述机制, DMS 将记忆的产生、使用、反馈和淘汰连接起来, 使记忆库能够随着 Agent 的任务执行过程不断调整, 从而形成持续的经验积累与演化。

3. 本次复现目标

本项目基于 AndroidWorld [2] 环境和开源视觉语言模型 (Qwen3-VL-8B-Instruct), 对 DMS 的核心算法逻辑进行工程复现, 并验证其相比于 Baselines 的优势。

具体而言, 本次复现主要完成以下工作:

- (1) **DMS 核心代码实现**: 依据原论文的算法逻辑, 实现分层记忆、效用计算、记忆选择以及动态淘汰等核心模块, 并完成各模块之间的数据流与调用关系。
- (2) **AndroidWorld 接入**: 将 DMS 记忆机制接入 GUI Agent 执行流程, 使 Agent 能够在任务执行过程中产生、调用和更新历史经验。
- (3) **Qwen3-VL-8B-Instruct 模型适配**: 使用 8B 级别的开源视觉语言模型作为底层 Agent 模型, 验证 DMS 的可行性。
- (4) **记忆进化效果验证**: 通过多轮 Trial 观察 Agent 任务成功率等指标的变化。
- (5) **记忆机制效果分析**: 通过不同 baselines 进行对比, 并结合 Token 消耗、执行步数和记忆库大小等指标, 分析 DMS 的优势。

后续章节首先介绍 DMS 复现工程的整体代码架构及核心模块实现, 然后基于 Android-World 开展实验, 对任务成功率、Token 消耗、执行步数和记忆库规模等指标进行统计, 并通过对比实验进一步分析 DMS 的实际效果。

二、 系统设计与工程实现

本部分介绍 Darwinian Memory System (DMS) 的代码设计与工程复现。系统以 Qwen3-VL-8B-Instruct 为底层 VLM, 以 AndroidWorld 为 GUI 交互环境, 在 Agent 执行链路中接入自己复现的 DMS 记忆模块。

代码上, 仓库由两部分构成: 官方 android_world/ (评测环境与任务集) 与自行实现的 DMS/ (记忆核心、Planner-Actor-Verifier、评测脚本), 二者同级放置。评测时在微软 Azure Linux 云服务器上启动 Live Android Emulator, 通过 ADB/gRPC 与环境交互。

VLM 不本地加载权重,经 OpenRouter API Key 调用云端多模态接口完成推理。考核推荐的 Qwen2.5-VL-7B-Instruct 在 OpenRouter 平台下架了,所以改用了 Qwen3-VL-8B-Instruct 进行测试。

1. DMS 处理流程与核心模块

整体采用与论文一致的 Planner - Actor 骨架: 同一 VLM 上切换记忆后端即可得到 Baseline A (无记忆)、Baseline B (静态追加) 与 DMS。

a) GUI Agent 执行流程

对每个 AndroidWorld 任务, Runner 在外层步进循环中反复调用 Agent。每步包括: (1) **观测**: 获取截图与无障碍树, 生成 UI 文本描述, 并在截图上叠加元素索引供 VLM 使用; (2) **规划**: Planner 依据总目标、屏幕状态与历史进度, 将任务分解为至多 5 条意图级子任务 $\langle \text{Precondition}, \text{Goal} \rangle$; (3) **检索与执行**: 对当前子任务, 以 Precondition 与 Goal 的双因子语义相似度检索记忆库。DMS 在命中且未触发 ε -mutation 或风险门控时复放所存 CodeAct 轨迹; 否则由 CodeAct 在多轮交互中生成 Python 代码, 调用 `open_app`、`tap`、`input_text`、`complete` 等工具, 直至子任务结束或预算耗尽; (4) **验证与回写**: 复放路径由 Verifier 结合截图判定子任务成败; 生成路径由 `complete/fail` 反馈判定。成功轨迹以 `llm_io` 格式写入记忆库; DMS 另经容量控制与生存剪枝维护库规模。全部步骤结束后, Runner 调用环境 `is_successful` 做任务级终判。Baseline A 关闭记忆检索与写入; Baseline B 保留检索复放与成功轨迹追加, 但不执行 ε -mutation、风险门控与达尔文式剪枝。

b) 分层记忆模块

记忆定义与处理在 DMS/core/types.py 和 bank.py 实现。记忆条目 $m = (p, \tau, meta)$: p 为意图键 $\langle pre, goal \rangle$, τ 为动作级轨迹 (存储在 JSON), $meta$ 含复用次数、失败次数、逻辑步等。索引存 SQLite, 轨迹存 JSON 文件, 实现 plan 与 action 解耦。检索采用双因子相似度 $sim(pre) \times sim(goal)$ (在 core/retrieval.py 实现), 超过阈值才召回。

c) 效用评估与记忆选择

Survival Value 在 DMS/core/survival.py 中实现:

$$S(m) = U(n) \cdot D(\Delta t, n) \cdot P(K)$$

其中效用随复用次数增长, 衰减随逻辑时间与半衰期变化, 可靠性随验证失败次数 K 下降。检索命中后以概率 ε 放弃复放 (core/mutation.py), 促使探索更优轨迹; 风险模块根据计划历史成败抑制高风险 plan (core/risk.py)。

d) 动态淘汰机制

容量调节与修剪在 `core/prune.py`: 当库规模超过 $C_{\min}$ 时, 按 $S(m)$ 降序曲线用 Elbow 估计应保留的主体与长尾; 长尾条目删除。若肘点处整体质量仍高, 则小步扩张容量上界。配合任务失败撤条与失败计数, 使低价值记忆在后期被抑制或淘汰。

e) AndroidWorld 与 VLM 接入

`runners/run_androidworld.py` 通过 AndroidWorld 的 env launcher 连接 Live Emulator, 将 DMS Agent 挂到官方 Agent 接口上逐步交互。`models/qwen_vl.py` 以 OpenAI 兼容 HTTP 客户端访问 OpenRouter (或 DashScope), 默认模型为 Qwen3-VL-8B-Instruct。系统的超参集中在 `configs/default.yaml`, 这些参数大多数按照论文附录设置; 实验的指标 (SR、步数/Token、Memory Size、MRR 等) 由 `metrics/` 文件夹的脚本按 trial 汇总并出图。

2. 机制映射

本节将论文机制映射到具体代码实现, 按两个维度整理:

- **表1:** 论文机制 → 核心代码文件及实现要点
- **表2:** 工程目录 → 模块功能说明

表 1: Darwinian Memory System 核心机制与复现代码映射

论文机制	主要实现位置	实现要点
Memory Construction $m = (p, \tau, s_{\text{meta}})$	<code>core/types.py</code> <code>core/bank.py</code>	定义 Plan/轨迹/记忆条目; SQLite 索引 + traj 落盘; 过滤过短轨迹
Dual-Factor Retrieval $\text{sim}(\text{pre}) \cdot \text{sim}(\text{goal})$	<code>core/retrieval.py</code>	双嵌入余弦检索; <code>top_k</code> / <code>min_score</code> 可配置
Survival Value $S = U \cdot D \cdot P$	<code>core/survival.py</code>	实现 $V_{\text{new}}, T_{\text{base}}, \mu, \beta, \gamma$; 效用 $\times$ 衰减 $\times$ 可靠性
Elbow Pruning $C_{\min}/C_{\max}$	<code>core/prune.py</code>	肘点裁剪与扩容; 规模 $\leq C_{\min}$ 时不修剪
ε -Mutation 更优轨迹替换	<code>core/mutation.py</code>	按 ε 探索; 成功且更短则原地进化

续下页

表 1: Darwinian Memory System 核心机制与复现代码映射（续）

论文机制	主要实现位置	实现要点
风险/声誉抑制 T_i	core/risk.py	贝叶斯全局失败率； 计划抑制阈值判决
闭环调节 query-commit-prune	core/regulate.py	DarwinianMemorySystem 循环； 检索、回流、计分、修剪
Planner-CodeAct (对照论文 PA 流水线)	agent/planner.py agent/codeact_agent.py agent/codeact_tools.py agent/codeact_exec.py agent/codeact_prompts.py	意图级分解 + 写 Python 调用工具执行
Audit-on-Use Verifier	agent/verifier.py agent/codeact_agent.py	主要在记忆回放后核查； 生成路径信任 Actor complete
对照系统 A / B / DMS	agent/baselines.py	A 无记忆；B 静态追加； DMS 完整自调节后端
评测协议 SR / Token / Mem / MRR	runners/run_androidworld.py metrics/round_metrics.py metrics/plot_metrics.py	多 Trial 评测； 指标汇总与曲线绘制

表 2: 复现工程主要模块与文件功能一览

路径	简要功能
core/types.py	核心数据结构：Plan、轨迹步、记忆 meta 与入库过滤
core/bank.py	记忆库：增删改查、向量字段、轨迹读写
core/retrieval.py	双因子检索与嵌入（BGE 等本地句向量）
core/survival.py	Survival Value $S(m)$ 公式
core/prune.py	Elbow 修剪与容量扩容
core/mutation.py	ϵ -mutation 与更优轨迹原地替换
core/risk.py	全局/计划级风险与抑制判决
core/regulate.py	DMS 闭环；加载 default.yaml
agent/planner.py	高层 Planner：目标 $\rightarrow$ 子任务 JSON

续下页

表 2: 复现工程主要模块与文件功能一览（续）

路径	简要功能
agent/codeact_agent.py	CodeAct×DMS 循环 + AW Agent 适配
agent/codeact_tools.py	点击/输入/open_app/complete 等
agent/codeact_exec.py	抽取并执行 VLM 输出的 Python 块
agent/codeact_prompts.py	Planner/CodeAct/Verifier 提示词
agent/app_names.py	App 名规范化（open_app 别名）
agent/verifier.py	复放后 Audit-on-Use 核查
agent/baselines.py	记忆后端 A/B/DMS 统一接口
agent/ui_desc.py	UI 树文本化与 SoM 提示
agent/json_utils.py	从 VLM 输出稳健抽 JSON
models/llm_protocol.py	多模态 LLM 协议（predict_mm）
models/qwen_vl.py	OpenRouter/OpenAI/DashScope HTTP 客户端
configs/default.yaml	Survival/容量/检索/mutation/风险/VLM 超参
runners/run_androidworld.py	评测主入口：任务循环、记分、写结果
runners/task_suites.py	test/min/preferred/full 套件定义
metrics/round_metrics.py	SR/SRR/MRR/Token/Memory 统计
metrics/plot_metrics.py	由 summary.json 绘对照曲线
scripts/_common.sh	环境变量与 DETACH/nohup
scripts/start_emulator.sh	启动/重置模拟器
scripts/run_test.sh	单任务冒烟
scripts/run_min.sh	Minimum（默认 DMS）
scripts/run_min_baselines.sh	Minimum 上 A/B
scripts/run_preferred.sh	Preferred（默认 DMS）
scripts/run_preferred_baselines.sh	Preferred 上 A/B
scripts/run_preferred_all.sh	Preferred 顺序 DMS→B→A
scripts/run_full_split.sh	Full Split（默认 DMS）
scripts/run_full_split_baselines.sh	Full Split 上 A/B

3. 系统启动

本复现在 Linux + KVM 上以 Live Android Emulator 为评测环境, VLM 经 OpenRouter 远程调用。实验机需支持 /dev/kvm, 安装 Python 3.11+、Android SDK 及 AVD Android-WorldAvd, 并配置 OPENROUTER_API_KEY。默认 VLM 为 Qwen3-VL-8B-Instruct, 检索嵌入为 BAAI/bge-small-en-v1.5。

一键脚本位于 DMS/scripts/: 先通过 start_emulator.sh 启动模拟器, 接着运行 run_*.sh, 跑出的记忆和结果会在 results/文件夹。详细步骤见 README.md, 脚本说明见表 2。

三、 实验与结果分析

1. 实验设置

本节基于前述 DMS 工程实现与 AndroidWorld 环境, 在微软 Azure 云服务器 (Standard D4as v5) 上开展实验。实验从任务成功率、执行效率及记忆库动态变化三个维度进行系统评估, 分析 DMS 的优势。

实验以 AndroidWorld 为 GUI Agent 交互环境, 采用 **Qwen3-VL-8B-Instruct** (经 OpenRouter API 调用) 作为底层视觉语言模型, BAAI/bge-small-en-v1.5 作为索引用的嵌入模型; 执行器为独立实现的 Planner-CodeAct 管线, 记忆轨迹以 llm_io (prompt / thought / code) 形式落盘, 复放时重跑历史代码。受时间与 API 成本限制, 我们未跑满完整 Split, 而采用下列套件, 进行 5 轮 Trial (轮次间记忆库连续保留, 固定随机种子以保证任务列表可复现):

- **Preferred:** 覆盖 ANDROID_FAMILY 全部真实 App, 每个 App 采样至多 2 个任务; 因部分 App 可评任务不足 2 个, 实际共 **34** 个任务 $\times$ 5 轮。

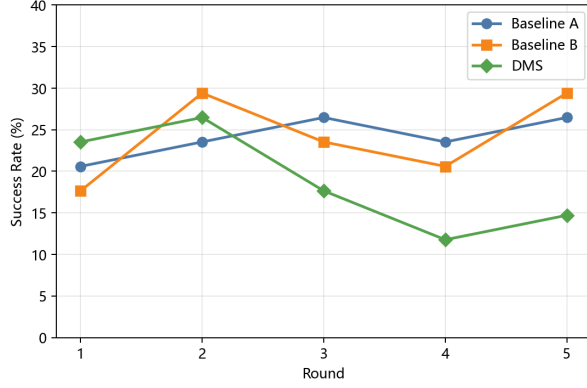
实验包含以下三种记忆机制配置:

- **Baseline A:** 不使用历史记忆, 仅根据当前任务和界面状态进行决策 (零样本)。
- **Baseline B:** 保存历史经验并在后续任务中检索复用, 静态追加、不进行动态淘汰。
- **DMS:** 采用本文复现的检索复放、Survival 效用评估与容量剪枝、风险门控及 ϵ -mutation 等自调节机制。

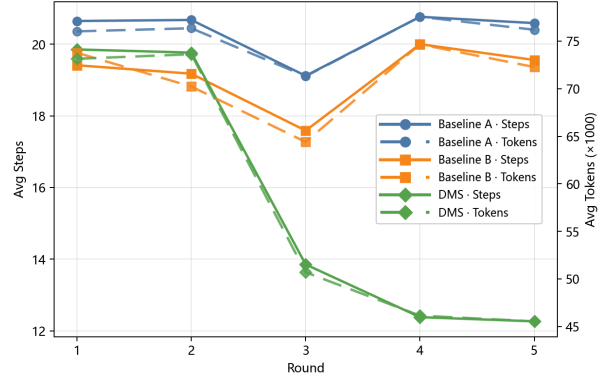
主要评价指标包括任务成功率 (SR)、成功保持率 (SRR)、记忆复用率 (MRR)、平均 Token 消耗、平均任务执行步数以及记忆库规模 (含峰值规模与 DMS 剪枝条数)。每轮结果落盘为 summary.json / round_metrics 与逐任务 task_results.json。

2. 主要对比结果与分析

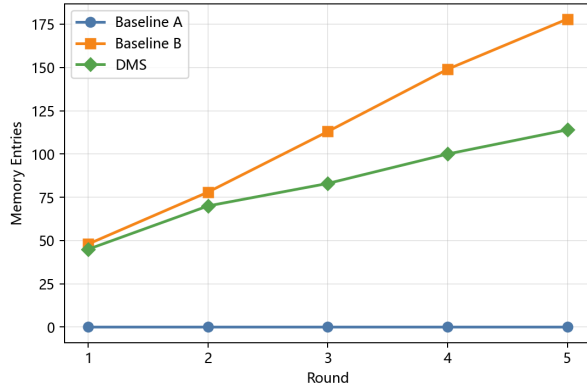
本节从任务成功率 (SR)、Token 消耗与执行步数、记忆库规模三个维度对比三种记忆机制在 Preferred (34 任务) 上的整体表现。



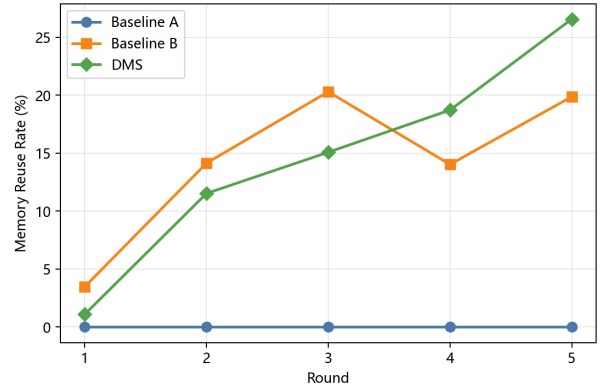
(a) 任务成功率随 Trial 演变



(b) Token 消耗与执行步数



(c) 记忆库规模变化



(d) MRR 指标演变

图 1: 不同记忆机制下 Agent 的主要对比结果

Preferred 套件共 34 任务、5 轮 Trial，曲线见 SR、效率、记忆规模与 MRR 四图。

任务成功率 SR。 Baseline A 无记忆，SR 在 21%–26% 间波动。Baseline B 有记忆、不剪枝，Round 2 与 Round 5 均达 29.4%，整体优于 A，说明记忆复用在本设定下确实有帮助。DMS 前两轮与 B 接近 (23.5%、26.5%)，但 Round 3 起持续下滑至 11.8%–14.7%，未能复现 DMS 优于静态记忆 Baseline 的结果。

平均步数与 Token 消耗。 三方法步数与 Token 走势基本一致，二者近似正相关。B 相对 A 略省；DMS 自 Round 3 起优势最明显，平均步数降至约 12，Token 降至约 4.6 万，效率优于 A 与 B。

记忆库规模。 纵轴为记忆条数，非存储空间。A 恒为 0；B 由 48 增至 178；DMS 由 45 增至 114，增速更慢，与动态剪枝有关系。

记忆复用率 MRR。A 恒为 0；B 与 DMS 复用率均高，DMS 逐轮上升，Round 5 达 26.6%，略高于 Baseline B。

3. gap 分析与解决

实验过程中我们观察到我们所用的 8B 模型和论文的 72B 模型在测试时有较大差别，本小节对这些差别进行了分析并且给出了一些解决方案。

a) 模型规模的影响（Gap 分析）

模型规模对系统表现的影响体现在多个层次。

(1) **任务成功率差距巨大**。原论文采用的大规模 VLM(如 72B 级参数)在 AndroidWorld 完整 Split 上可达到约 40%–70% 的 SR；本实验使用 Qwen3-VL-8B 时，在 Preferred 套件上首轮 SR 仅约 10%，与大规模模型相差大。该差距具有明显的难度分层：Easy 有少部分能做出来，Medium 和 Hard 任务几乎都做不出来。当前规模模型已完成部分简单 GUI 操作，但在跨 App、多步约束的复杂任务上规划与执行能力严重不足。

(2) **记忆质量低**。系统的 Planner 与 Actor 均由 8B 级模型承担，过小的模型导致角色能力差，从以下几个层面影响记忆质量。(i) **规划层面**：Planner 常将总目标拆分为不当或冗余的子任务 $\langle Precondition, Goal \rangle$ ，出现空计划、过早声明 `task_done` 或与界面状态不符的意图；(ii) **存储层面**：弱模型生成的 CodeAct 轨迹本身错误率较高，若仍被判定为子任务成功，会误写入记忆库，形成记忆噪声；(iii) **验证层面**：Verifier 对复放结果误判，使错误轨迹记忆获得重放资格，导致任务失败。上述因素共同导致记忆库有条目但是低信噪比，MRR 长期维持在 1% 量级，SR 曲线难以呈现论文中的明显上升趋势。

综上，小模型不仅拉低 SR 上限，更会恶化规划、存储、验证各环节质量削弱甚至逆反记忆的优势。

b) Gap 应对策略

针对上述差距，系统从**记忆表示**、**感知与执行胶水层**、**提示约束**三个方向进行缓解。

(1) **记忆表示**。早期实现将单步 JSONAction 直接入库，信息不足，agent 难以复现。现改为 `llm_io` 格式：每条轨迹保存 `prompt`、`response_code` 与 `response_raw`；复放时重执行 `response_code`，由工具层映射为 JSONAction，而非盲放历史坐标。入库前以有效 CodeAct 轮数 $|\tau| > 1$ 过滤，并保留子任务级成功轨迹。

(2) **感知与执行胶水层**。(i) Set-of-Mark 为可交互控件叠加序号框，引导模型优先 `tap_by_index`；(ii) 仅暴露 `open_app`、`tap_by_index`、`input_text`、`complete/fail` 等白名单 API；(iii) 应用名规范化，将输出映射到 AndroidWorld 注册名；(iv) 执行前校验 `index` 合法性，越界直接返回错误日志。

(3) **提示词约束**。8B 模型容易规划过碎或在执行时偏离任务原文，因此在 Planner、CodeAct 与 Verifier 三处加入轻量约束。Planner 每次仅输出 1-3 个子任务，并在 Goal 中保留姓名、号码、文件名等字面信息；打开应用时明确要求调用 `open_app`，避免在桌面盲搜，且不再重复 Progress 中已成功的步骤。CodeAct 每轮只生成一段 Python，优先基于当前界面索引操作，子目标完成即 `complete()`，以减少无效探索。Verifier 主要用于记忆复放后的核查，对打开应用类子目标采用较宽松判定，界面已进入对应 App 即可通过。

上述策略有效提高了模型的 SR。在 Preferred 套件上，首轮 SR 由不足 10% 提升至高于 20% (Easy: 85.7%, Medium: 25.0%, Hard: 0%)。

4. SR 复现异常的分析与原因推测

DMS 在 Preferred-34 上 MRR 和效率指标上有优势，在中后期步数、Token 都明显下来了，但 SR 从第 3 轮起一直低于 Baselines，跟论文里的预期对不上。结合实现细节和跑出来的曲线，我们推测主要有三点原因：

- **模型和执行栈本身就有差距**。我们用的是 Qwen3-VL-8B，配自研的 CodeAct 流水线；论文是 72B 的骨干模型，执行栈也不完全一样。8B 在规划、判对错、写代码这几步上容错都偏低，一旦记忆里存的轨迹有偏，复放时更容易直接把题做挂。
- **记忆相关的校验链路没完全搭齐**。比如 Verifier 主要卡在复放路径，生成路径、多轮校验、风险统计和入库/修剪细节，跟原文附录的算法有所出入。
- **错误记忆的副作用盖过了收益**。从结果看，DMS 中后期复用记忆明显增多，效率也确实是在涨，但 SR 同步往下掉。更像是坏记忆被反复调用，把原本重新生成能救回来的情况也带偏了。影响记忆质量的因素很多——检索阈值、校验强度、入库门槛、跨 App 泛化，还有 8B 模型本身的噪声，都可能掺一脚。我们在 Gap Analysis 里解决了一部分，但因为时间关系，还有一些情况没研究透。

参考文献

- [1] H. Mi, Y. Feng, W. Lu, S. Cao, J. Li, Y. Li, X. Zhang, H. Luo, S. Peng, H. Cui, T. Tian, J. Fang, H. Chai, and N. Tan, “Darwinian memory: A training-free self-regulating memory system for gui agent evolution,” 2026. [Online]. Available: <https://arxiv.org/abs/2601.22528>
- [2] C. Rawles, S. Clinckemaillie, Y. Chang, J. Waltz, G. Lau, M. Fair, A. Li, W. Bishop, W. Li, F. Campbell-Ajala, D. Toyama, R. Berry, D. Tyamagundlu, T. Lillicrap, and O. Riva, “Androidworld: A dynamic benchmarking environment for autonomous agents,” 2025. [Online]. Available: <https://arxiv.org/abs/2405.14573>